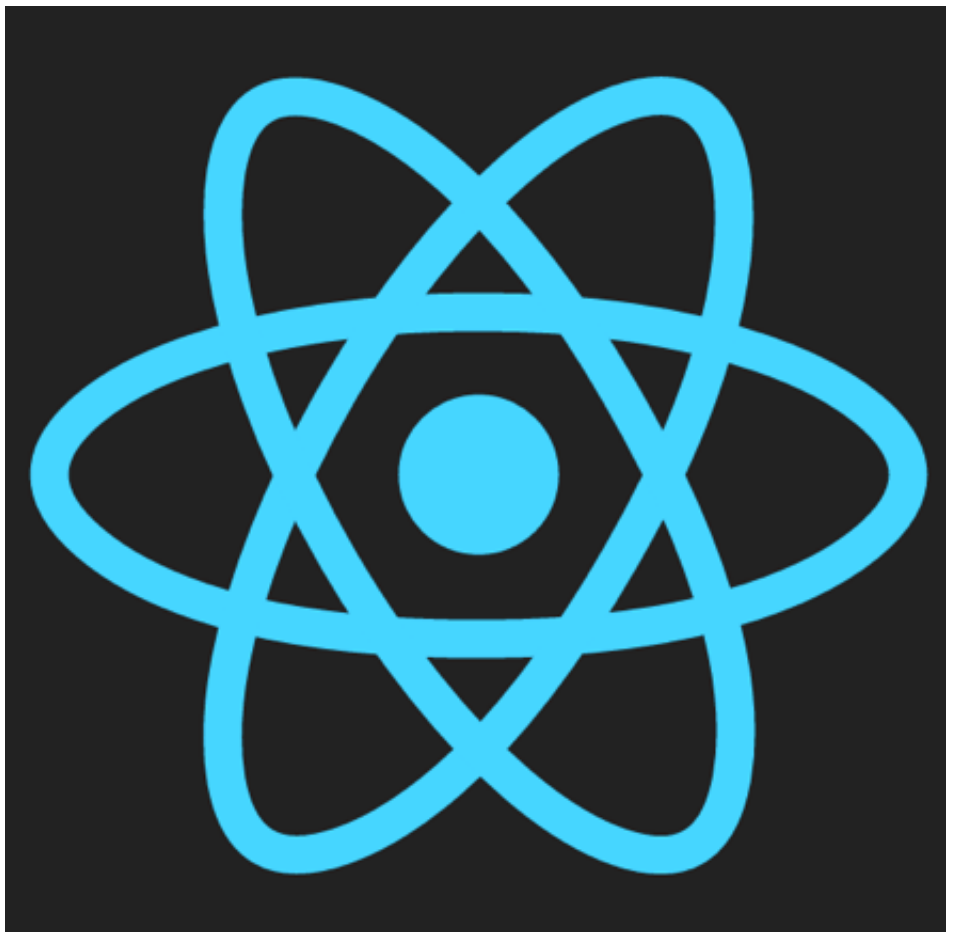


Advanced React Development

copyright 2024, Chris Minnick

version 1.3.0,
March 2024



Introduction

- Objectives
 - Who am I?
 - Who are you?
 - Daily Schedule
 - Course Schedule and Syllabus



The Big Picture

Some of the Topics
Covered in this Course

Redux

Component
Lifecycle

React Router

Server-side
Rendering

Code
Splitting

Micro-
frontends

Performance

Security

The Labs

- Instructions: [advanced-react-labs.pdf](#)
- Working in small groups.
- Let me know if something doesn't work.
- If you finish a lab early:
 - try lab challenges,
 - help other students,
 - do additional reading,
 - take a break.

Bootstrapping React Apps

- Create React App is dead.
- Exploring Alternatives
- Converting from CRA to Vite

React Frameworks

- Next
- Remix
- Astro
- React Query

A Deeper Dive into React Fundamentals

React Component Lifecycle

Mounting

The component renders in the browser.

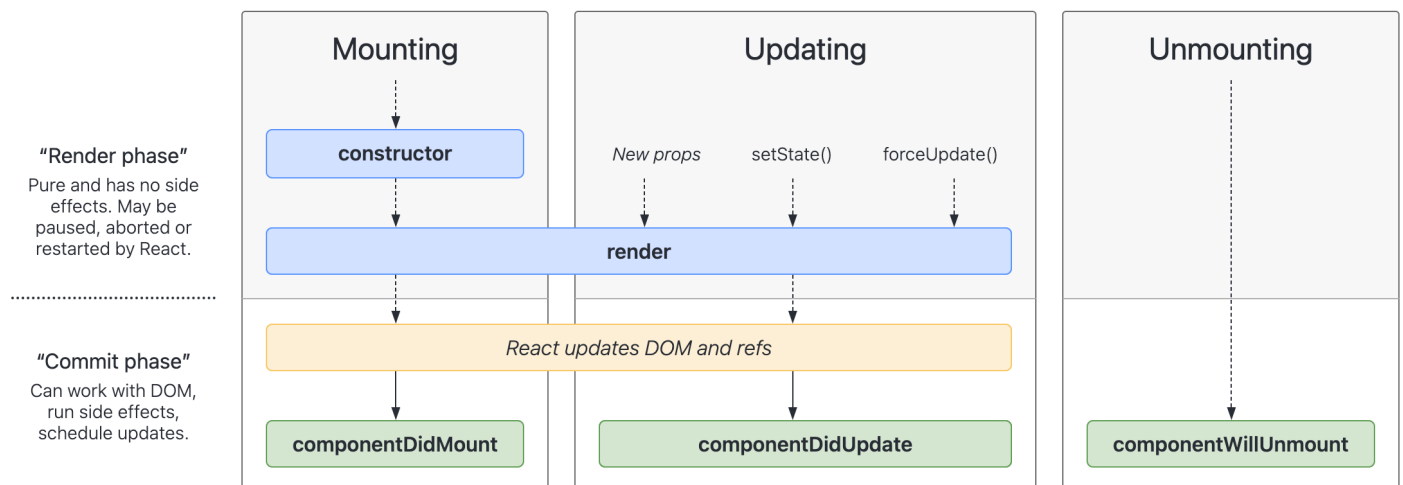
Updating

The component receives new props or sets its state.

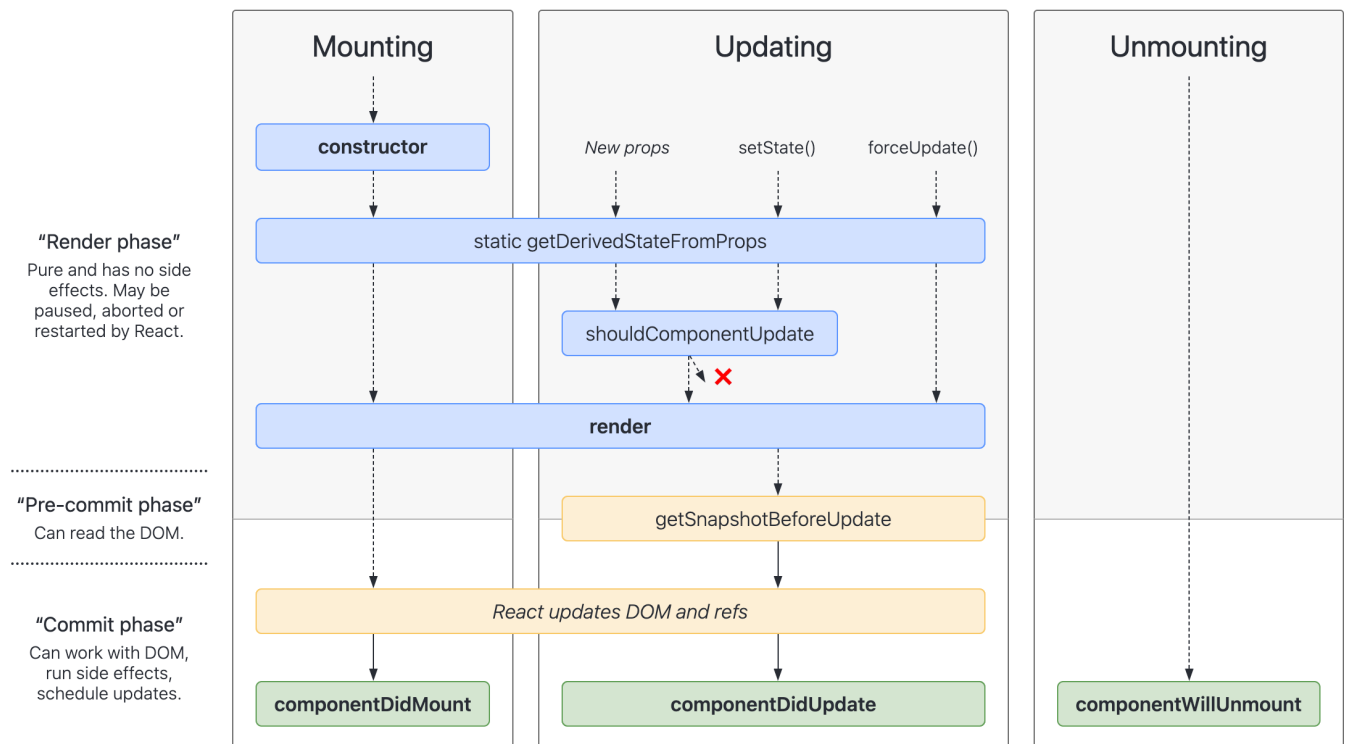
Unmounting

The component is removed from the browser.

React Lifecycle



React Lifecycle



Introduction to Component Lifecycle



What is Component Lifecycle?



Importance in React Development

Mounting Phase

`constructor()`

`static getDerivedStateFromProps()`

`render()`

`componentDidMount()`

Updating Phase

`static getDerivedStateFromProps()`

`shouldComponentUpdate()`

`render()`

`getSnapshotBeforeUpdate()`

`componentDidUpdate()`

Unmounting Phase



`componentWillUnmount()`

Called just before the component is removed from the DOM



Best practices

Use for cleaning up side-effects initiated in other lifecycle methods (timers or event listeners).

Error Handling Phase



`static getDerivedStateFromError()`



`componentDidCatch()`

Lifecycle Methods in Hooks



`useEffect()`



`useState()`

Lifecycle Best Practices



Avoid Heavy Computations in render Method: Offload heavy calculations to lifecycle methods to avoid unnecessary re-renders.



Use `componentDidMount` for API Calls: Fetching data in this method ensures that the component has mounted before attempting network requests.



Optimize Child Components: Utilize `React.memo` or `PureComponent` for functional and class components, respectively.



Cleanup in `componentWillUnmount`: Always remove event listeners, cancel network requests, and invalidate timers before a component unmounts.

Class Components



JavaScript classes that extend `React.Component`



Can hold and manage state.



Have access to lifecycle methods.



Must have a render method.



Useful for more complex components or components that require direct access to lifecycle methods.

State Management with `setState()` in Class Components

- Takes an object or a function that returns an object as its argument.
- Merges the object with `this.state`
- Is asynchronous
- If you pass a function, the function always receives the latest state.
- If the new state depends on the previous state, always use a function



Function Components



Basically, the `render()` method from a class component.



Cannot access lifecycle methods.

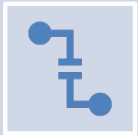


Cannot maintain its own state.



Access to lifecycle and state from functions is done using Hooks.

Hooks



Functions provided by React that give functions access to internal workings of React.



Useful for simulating component lifecycle and associating state with function components.

How to think about Hooks

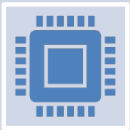
Hooks seem magical at first.

Using them correctly requires a deeper understanding.

Important Points:

- Hooks store data outside of a function.
- A Hook is associated with an “instance” of a function component

How to think about Hooks (cont.)



Function components don't technically have instances, and they don't technically render.



A function component is just a function that returns a piece of the UI when it's invoked.



Unless you store data outside of the function and access it when the function runs, functions have no way of persisting data between invocations.

Understanding useState



The useState hook sets a persistent value in React and associates it with the “instance” of the function component.



Every time the function runs, each call to useState() gets its latest value from the React library (except the first time, when the value passed to useState() initializes the value).



useState() returns an array with 2 elements

the latest value
a function for updating the value
• this is called the “state dispatch function”

Understanding useEffect



The `useEffect` takes a function as its first argument, and an array of dependencies as its second argument.



Each time the function component renders, `useEffect` compares the previous values of the dependencies with the new values.

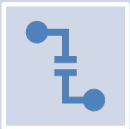


If the new values of any dependencies have changed, the function passed to `useEffect` runs.

Understanding useEffect (cont.)



If no dependency array is passed, the function passed to `useEffect` will run every time the function component “instance” renders.



If an empty array is passed, the effect will only run on the first invocation of the “instance” of the function component.



If the effect function returns a function, that function will run when the component is “unmounted.”

useCallback vs. useMemo

- `useCallback`: caches a function.
 - `useCallback(fn, dependencies)`
 - React will **return** the cached function with each render, provided that the dependencies haven't changed.
- `useMemo`:
 - `useMemo(()=>calculateValue, dependencies)`
 - React will **call** the function on first render, and return the cached value on subsequent renders, unless the dependencies have changed.

useDeferredValue

- Can be used with `<Suspense>` to show stale content while fresh content is loading.
- Requires use of Relay (GraphQL), Next.js, or Lazy-loading components.
- <https://react.dev/reference/react/useDeferredValue>

useTransition

- Lets you update the state without blocking the UI
- Allows you to mark a state update as a non-blocking transition

useLayoutEffect

- A version of useEffect that fires before the browser repaints the screen.
- Use sparingly - can hurt performance.
- Demo:
<https://react.dev/reference/react/useLayoutEffect>



Lab: React Fundamentals

- Add new features to an existing real-time chat application

React Router



Route Configuration

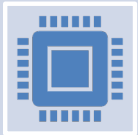


Navigation



Protected Routes

Introduction to React Router



What is React Router?

dynamic routing library built on top of the React framework. React Router uses a component-based structure to define routing rules.



Why is it important?

SPA Navigation
Declarative routing
Code splitting

Router Component

- Wraps around the Routes component
 - data APIs
 - version 6.4+
 - createBrowserRouter
 - recommended for browser-based apps
 - createHashRouter
 - createMemoryRouter
 - createStaticRouter
 - pre-6.4 (still supported)
 - <BrowserRouter>
 - <MemoryRouter>
 - <HashRouter>
 - <NativeRouter>
 - <StaticRouter>

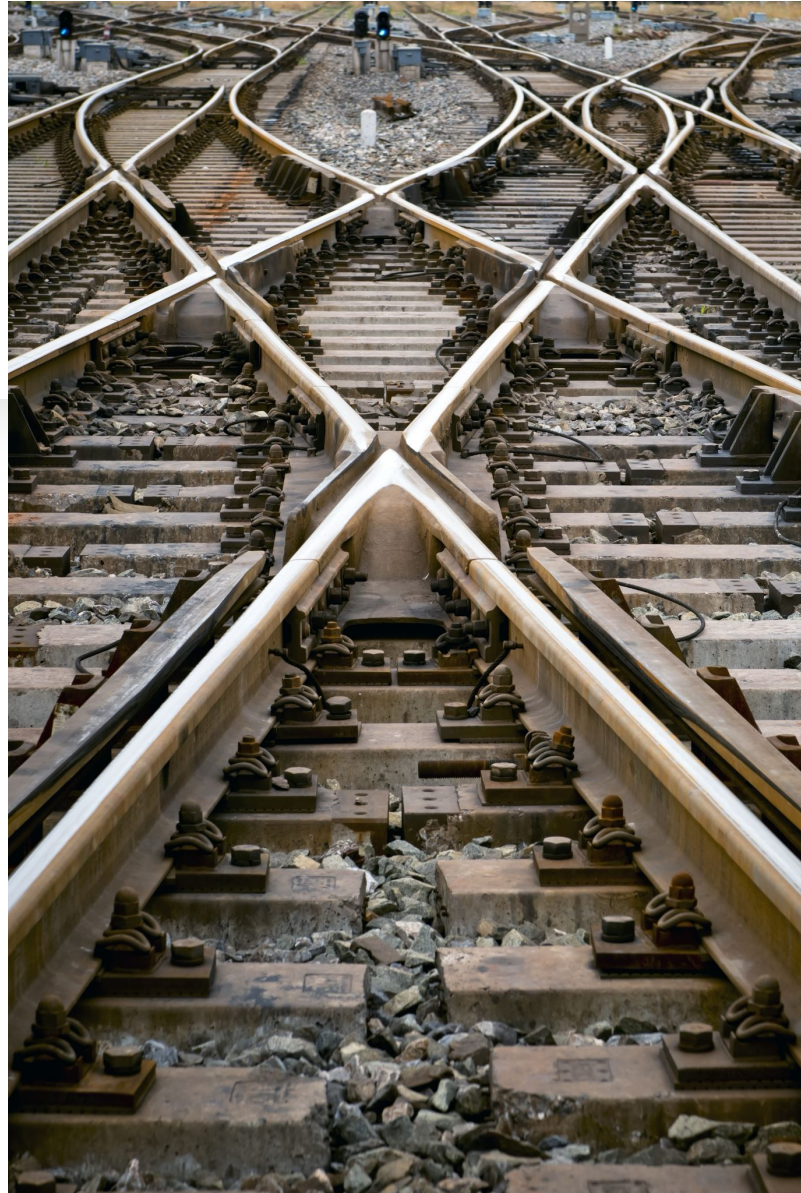
Routes Component

- Wraps around a group of Route components.



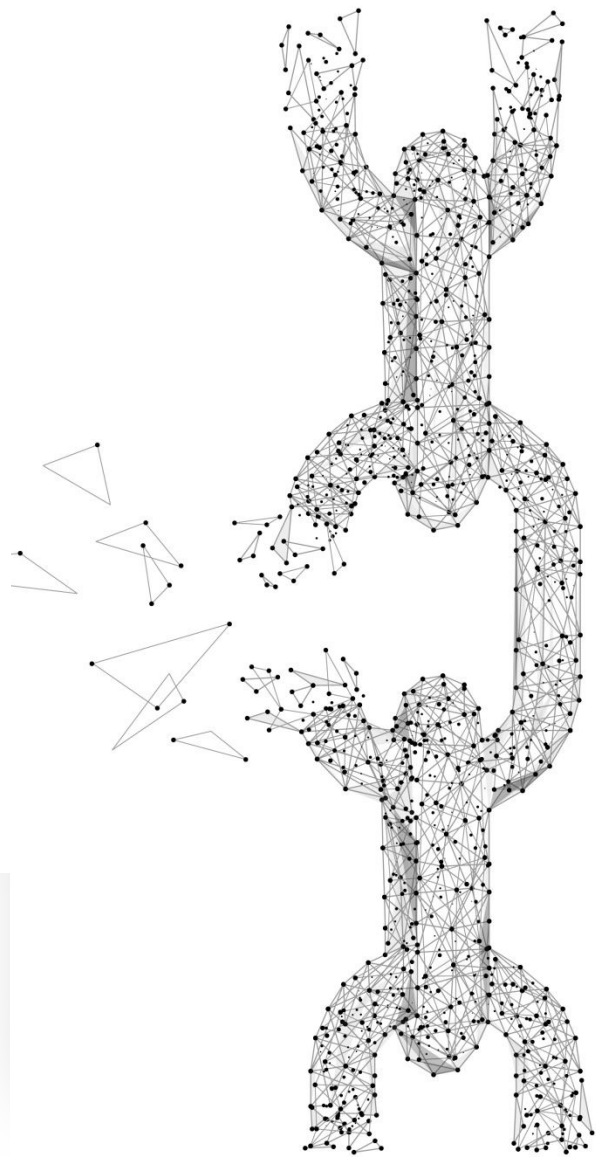
Route Component

- Matches paths to elements
- Must be a direct child of a Routes component



Link and NavLink Components

Explanation and
differences



redirect

- Redirects to a new location.

```
import { redirect } from "react-router-dom";  
const loader = async () => {  
  const user = await getUser();  
  if (!user) {  
    return redirect("/login");  
  }  
  return null;  
};
```

Dynamic Routing

- Route parameters
 - define routes with variable parts
 - `/users/:userId` matches `/users/123`, `/users/abc`, etc
 - Query Strings and URL parsing
 - `/search?query=react` allows the component to access query

Hooks in React Router

useHistory

- provides access to the history object
- push(path)
- replace(path)

useParams

- `const {userId} = useParams();`

useLocation

- Provides access to the location object
- `const location = useLocation();`
- `return <div>Current path: {location.pathname}</div>;`

Outlet Components

- Used in a parent route element to render its child route elements.

Nested Routes

- Create a component that renders an Outlet
- Nest Route elements inside the Outlet route.

```
function Dashboard() {  
  return ( <Outlet /> );  
}  
function App() {  
  return (  
    <Routes>  
      <Route path="/" element={<Dashboard />}>  
        <Route path="inbox" element={<Inbox />} />  
        <Route path="tasks" element={<Tasks />} />  
      </Route>  
    </Routes>  
  );  
}
```

Implementing Protected Routes

```
import { Navigate, Outlet } from 'react-router-dom'
const PrivateRoutes = () => {
  let auth = {'token':false}
  return (
    auth.token ? <Outlet/> : <Navigate to='/login' />
  )
}
function App() {
  return (
    <Routes>
      <Route element={<PrivateRoutes />}>
        <Route path="inbox" element={<Inbox />} />
        <Route path="tasks" element={<Tasks />} />
      </Route>
      <Route path="/login" element={<Login />} />
    </Routes>
  );
}
```

Alternatives to React Router

- wouter
- Next JS
- React Location

Next.js

- Folder-based routes (App router)
- Server-side Components
- Streaming
- Much more!

Lab: Routing and Authentication

- Create a signup component and a login component
- Use react router to create the following routes:
 - /login
 - /signup
 - /logout

Redux: Introduction to Redux



WHAT IS REDUX?



WHY USE REDUX?

Redux: Redux Architecture



Basic Architecture

Store
Actions
Reducers



Flow of Data

Action -> Store -> Reducer ->
New State -> View Update

Redux: Actions



Information payloads that send data from the application to the store.



Plain JavaScript objects containing a type property to indicate the type of action being performed.



Optionally contain additional data.

Redux: Dispatching Actions

- How to Dispatch

```
const increment = () => {  
  return {  
    type: 'INCREMENT',  
  };  
};
```

```
// Dispatching Action  
dispatch(increment());
```

Dispatching Async Actions (Thunk)

```
const fetchData = () => {  
  return (dispatch) => {  
    fetch('https://api.example.com/data')  
      .then((response) => response.json())  
      .then((data) => dispatch({ type:  
'FETCH_DATA_SUCCESS', payload: data })))  
      .catch((error) => dispatch({ type:  
'FETCH_DATA_ERROR', payload: error }));  
  };  
};  
  
// Dispatching Async Action  
dispatch(fetchData());
```

Redux: Reducers



Functions that determine how the state will change in response to an action.



Receive the current state and an action as arguments, and return a new state.



Must remain pure functions – no side effects or mutations.

Redux: Store

- Single source of truth.
- Holds the state of the entire application.
- Created using the createStore method.

Redux: Redux Middleware

- What is Middleware?
- Common Middleware Examples

Redux: Redux Thunk

- Async Actions
- Thunk Middleware

Redux: Redux Toolkit

- What is Redux Toolkit?
 - Official toolset for efficient Redux development
- Benefits
 - Simplifies common use cases
 - Provides good defaults for store setup
 - Write “mutative” immutable update logic
 - Create “slices” automatically

Redux: createAction

- Generates action creators based on the given action type.

Basic Example:

```
const increment =  
createAction('counter/increment');
```

Dispatch:

```
dispatch(increment());  
or dispatch(increment(5));
```

Redux: createReducer

- Simplifying Reducer Creation

```
const myReducer =  
createReducer(initialState, {  
  [actionType]: (state, action) => {  
    // reducer logic  
  }  
});
```

createReducer with Payload

```
const add = createAction('ADD');

const counterReducer =
  createReducer(0, {
    [add]: (state, action) => state +
    action.payload,
  });
```

createSlice

```
import { createSlice } from '@reduxjs/toolkit';

const counterSlice = createSlice({
  name: 'counter',
  initialState: 0,
  reducers: {
    increment: (state) => state + 1,
    decrement: (state) => state - 1,
  }
});

const { increment, decrement } = counterSlice.actions;

dispatch(increment());
```

Redux: createSelector

- What is createSelector?
 - creates memoized selectors
- Benefits of Using
 - helps to optimize re-rendering and re-computation by caching the result of complex calculations based on the Redux state.

```
const selector = createSelector(  
  [inputSelector1, inputSelector2],  
  (output1, output2) => {  
    // perform computation  
  }  
);
```

Redux: Use with React

- react-redux
 - Official React bindings for Redux
 - <Provider>
 - useSelector
 - useDispatch

Lab: Implementing Redux

- Convert the social media app to use Redux



Micro Frontends

- Micro Frontends decompose frontend applications into independent units in the same way that microservices decompose the back end.
- Gives teams the ability to independently develop, test, and deploy each micro frontend.

Introduction to Micro-frontends

- Why consider using them?
 - Deploy microfrontends independently
 - Use multiple frameworks on the same page
 - Lazy load code for improved initial load

React Micro Frontends

- Challenges:
 - Overhead: Each micro frontend needs to access the same libraries, CSS, routes
 - Can create massive problem of duplication of code and poor performance
 - Cross-cutting concerns: Each frontend needs to share authentication, for example.
 - Communication between micro frontends is more complex than between components in an app.

Common strategies for React Micro frontends

- Use a container application to handle rendering and routing between the micro frontends.
- Use a CDN to host shared frontend libraries and resources.

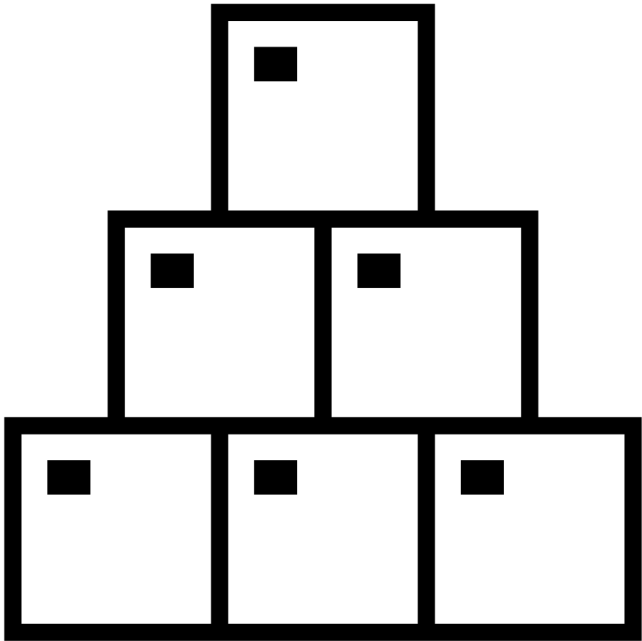
Shared State in React Micro Frontends

- Four strategies
 - Web workers
 - Props and callbacks
 - Custom events
 - Pub Sub library

Tools for Implementing Micro-frontends

- iFrames
- Web Components
- Server-Side Includes
- Single SPA (single-spa.js.org)
- Module Federation (Webpack 5)
- Tailor.js
- Pirial

Lab:
Micro
frontends with
Single SPA



Performance Optimization

Measuring Performance

- Lighthouse
- React DevTools Profiler
- `reportWebVitals();`

How Code Splitting Works

- Code Splitting is a feature supported by bundlers like Webpack and Browserify which can create multiple bundles that can be dynamically loaded at runtime. It helps in reducing the initial load time by splitting the application into smaller chunks which are loaded as they are needed.

Dynamic Imports

- The `import()` syntax is a function-like expression that allows loading an ECMAScript module asynchronously and dynamically.
- `import ( './SomeComponent' )`
- Import declarations (`import SomeComponent from './SomeComponent'`) is static. The imported module is evaluated at load time.

React.lazy() and Suspense

- React.lazy() lets you render a dynamic import as a regular component.
- ```
const SomeComponent = lazy(() => import(`./SomeComponent`));
```
- The component won't be loaded until it's rendered the first time.
- Wrap a lazy-loaded component with `<Suspense>` to specify what should display while the component is loading.

# Lazy & Suspense Example

```
const SomeComponent = React.lazy(() =>
import('./SomeComponent'));

function App() {
 return (
 <Suspense fallback={<div>Loading...</div>}>
 <SomeComponent />
 </Suspense>
);
}
```

# Route-based Code Splitting

- Delay loading components for Routes until those Routes are rendered.

```
function App() {
 return (
 <Router>
 <Suspense fallback={<div>Loading...</div>}>
 <Routes>
 <Route exact path="/" element={<Home />} />
 <Route path="/about" element={<About />} />
 </Routes>
 </Suspense>
 </Router>
);
}
```

# Performance Optimization in Class Components

- **shouldComponentUpdate()**. Returning false from this lifecycle method will skip render().
- **PureComponent**. Extend (instead of `React.Component`) to indicate that a component's output is wholly dependent on its props (it's "pure"). Render will be skipped if props are the same.

# Performance Optimization in Functions

- `useMemo` memorizes values. Use it to memorize the output of expensive calculations.
- `useCallback` memorizes functions. Use it to avoid unnecessary renders.
- `React.memo`: pass function components to `React.memo()` to cause them to skip re-rendering if the props passed are unchanged.

# Batching Updates

- Avoid extra renders by taking advantage of state's asynchronous nature.

```
const handleUpdate = () => {
 setState(prevState => ({
 ...prevState, count1: prevState.count1 + 1
 }));
 setState(prevState => ({
 ...prevState, count2: prevState.count2 + 1
 }));
}
```

- Starting with React 18, React automatically batches updates.

# Lazy Initialization

- Avoids recalculating or refetching initial state on every render.
- Instead of:
  - `const initialState = expensiveCalculation(props)`
  - `const [state, setState] = useState(initialState);`
- Do this:
  - `const getInitialState = () => window.localStorage.getItem('state');`
  - `const [state, setState] = useState(getInitialState);`

# Data Fetching Libraries / Hooks

- SWR
- React Query

# Web Workers

- Off-the-main-thread calculations

# Use Throttling to Limit Event Handlers

- Throttling is used to create a throttled function that can only call the func parameter once per every wait milliseconds.
- `import React, { useEffect } from 'react';`
- `import { throttle } from 'lodash';`
- <https://codesandbox.io/s/throttling-dr695r?file=/src/ThrottlingExample.js>

# Debouncing

- The Lodash debounce function returns a debounced function that will execute a function after X milliseconds pass since its last execution.
- Use for limiting API requests.
- <https://codesandbox.io/s/throttling-dr695r?file=/src/DebouncingExample.js>

# Introduction to Server-Side Rendering

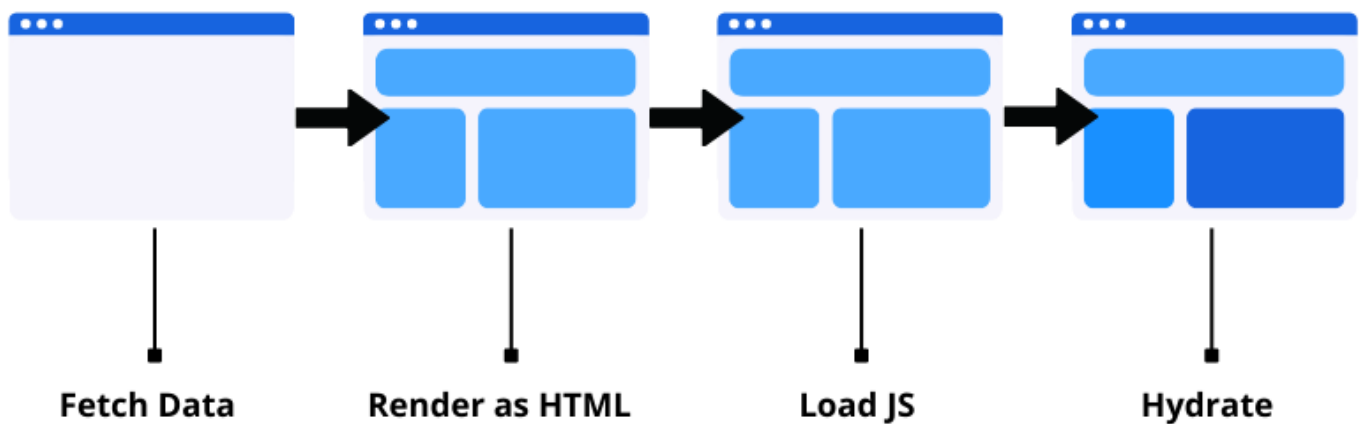
- What is SSR?
- Why is it important?

# Client-Side Rendering vs Server-Side Rendering

- Key differences
- Pros and Cons

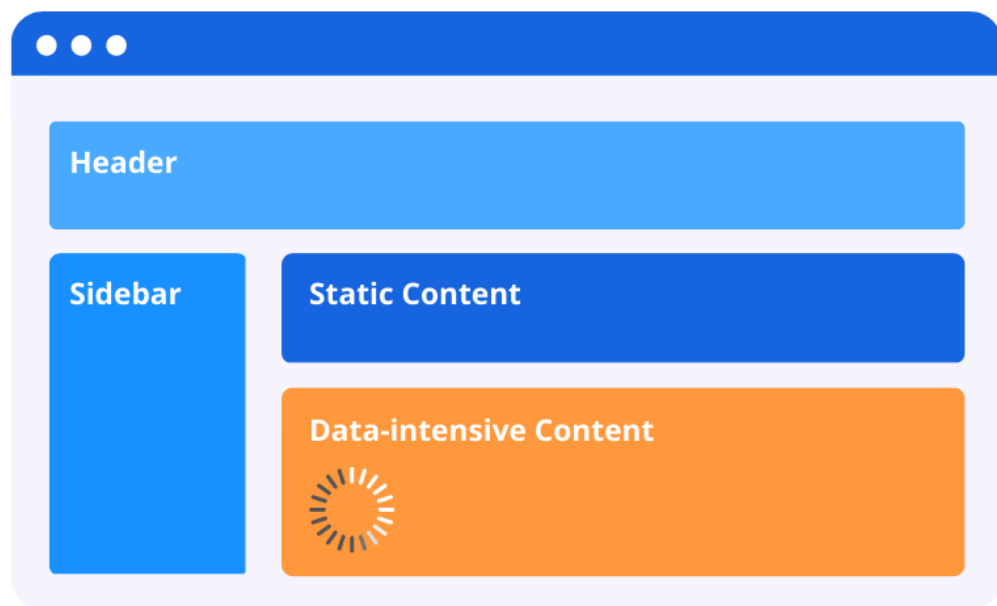
# How SSR Works (< React 18)

## Server-Side Rendering Before React 18



# How SSR works in React 18

## Server-Client Rendering with `<Suspense>`





## Introducing Server Components

# Tools for SSR in React

- Next.js
- Gatsby

# Advantages of SSR

- SEO benefits
- Faster initial page loads

# Drawbacks of SSR

- Server load
- Complexity

# Hydration

- What is Hydration?
  - Using client-side JavaScript to add application state and interactivity to server-rendered HTML

- How it works

```
import { hydrateRoot } from 'react-dom/client';
```

```
hydrateRoot(document.getElementById('root'), <App />);
```

- <https://codesandbox.io/s/hydrate-qyy2mr?file=/public/index.html>

# Lab: Performance Optimizations

---

# **SECURITY BEST PRACTICES**



# Use the Latest Version of React

- Always make sure you're using the latest version of React and its associated libraries. This ensures that you benefit from the latest security patches and updates.

# Escape User Input

- React automatically escapes content rendered with JSX to prevent Cross-Site Scripting (XSS) attacks. However, when you use methods like `dangerouslySetInnerHTML`, you're bypassing this protection. Use this method sparingly, and always sanitize any user input or content that might be rendered.

## **Avoid Inline Event Handlers with User Data**

- Do not inject user data directly into inline event handlers in JSX. It could create potential security risks if not properly sanitized.

## **Beware of Third-party Components**

- Not all npm packages or third-party React components follow best security practices. Always audit and review the code of third-party components before including them in your project.

## Use HTTPS

- Serve your React application over HTTPS to ensure that data exchanged between the client and the server remains encrypted and secure.

# Manage Secrets Properly

- Never hard-code sensitive information or secrets in your React code. Use environment variables or server-side configurations for this.

# Validate and Sanitize Prop Data

- If you're passing data to components via props, ensure the data is validated and sanitized, especially if the data originates from user input or external sources.

# Handle Redirections Carefully

- If your application includes redirections based on user input, validate the input to ensure that it doesn't lead to unintended or malicious URLs.

# Use Tokens for Authentication

- When implementing authentication, consider using JWT (JSON Web Tokens) or similar token-based methods, and store them securely. Avoid storing them in local storage due to potential XSS attacks; instead, use HttpOnly cookies.

# Keep Dependencies Updated

- Regularly update all your project's dependencies. This helps patch vulnerabilities that might exist in older versions of libraries you're using.

# Regularly Audit Dependencies

- Use tools like `npm audit` or `yarn audit` to identify and fix vulnerabilities in your project's dependencies.

# Implement Rate Limiting and Throttling

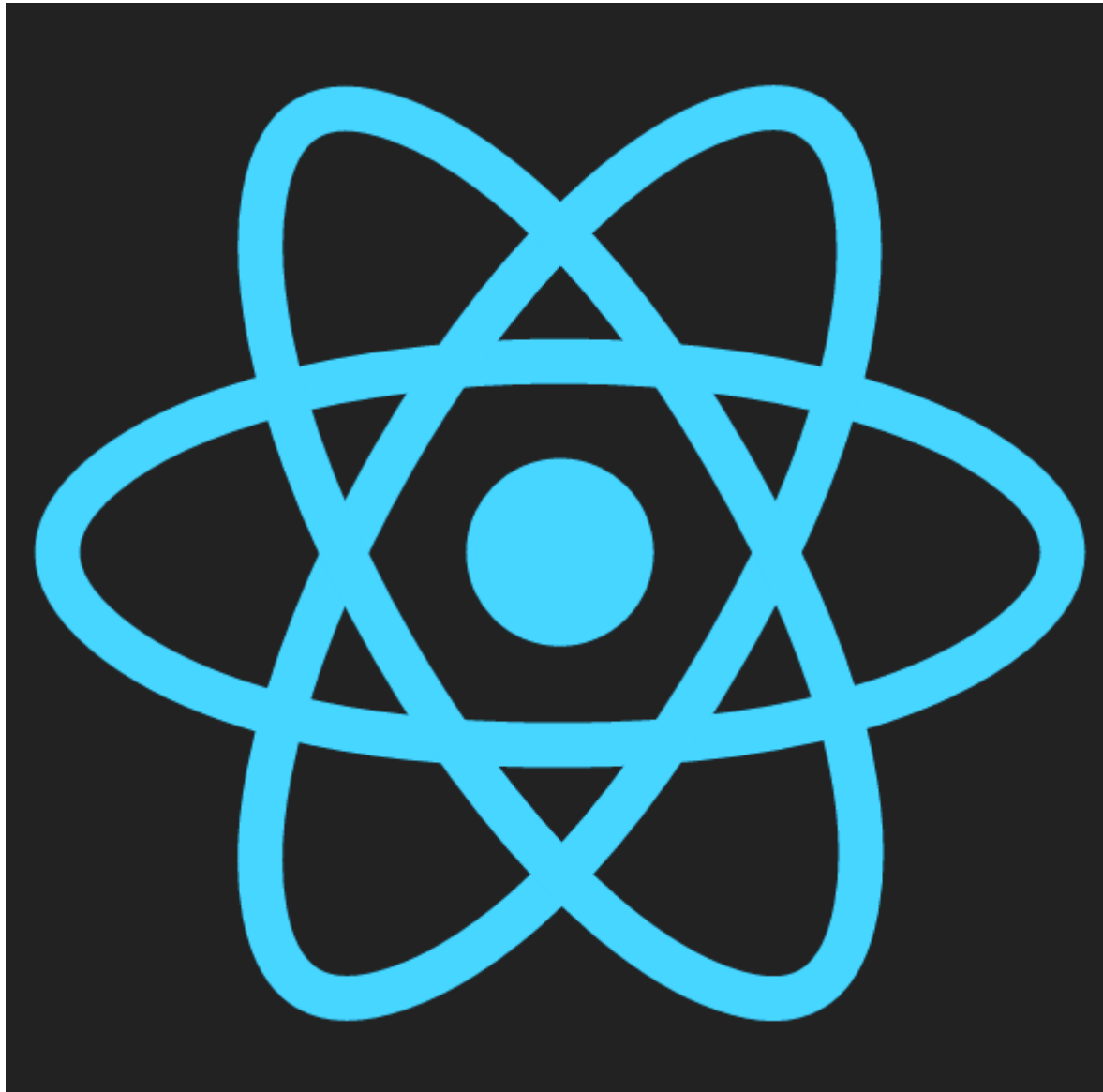
- If your React app communicates with an API or server, implement rate limiting to prevent abuse.

## Use Strong CORS Policies

- If your React app communicates with an API, ensure that Cross-Origin Resource Sharing (CORS) policies are correctly set up to limit which domains can access your resources.

Q&A

# Advanced ReactJS



Completed source code for all labs (for checking your work) can be found at:

<https://github.com/chrisminnick/advanced-react>

Version 1.3.0, March 2023  
by Chris Minnick  
Copyright 2024, WatzThis?  
[www.watzthis.com](http://www.watzthis.com)



## Table of Contents

|                                                      |           |
|------------------------------------------------------|-----------|
| <b>TABLE OF CONTENTS.....</b>                        | <b>2</b>  |
| <b>DISCLAIMERS AND COPYRIGHT STATEMENT .....</b>     | <b>3</b>  |
| DISCLAIMER.....                                      | 3         |
| THIRD-PARTY INFORMATION .....                        | 3         |
| COPYRIGHT .....                                      | 3         |
| HELP US IMPROVE OUR COURSEWARE. ....                 | 3         |
| <b>CREDITS.....</b>                                  | <b>4</b>  |
| ABOUT THE AUTHOR.....                                | 4         |
| <b>COURSE REQUIREMENTS .....</b>                     | <b>5</b>  |
| <b>LAB 01: REACT FUNDAMENTALS .....</b>              | <b>6</b>  |
| <b>LAB 02: ROUTING AND AUTHENTICATION .....</b>      | <b>7</b>  |
| <b>LAB 03: IMPLEMENTING REDUX .....</b>              | <b>8</b>  |
| <b>LAB 04: MICRO FRONTENDS WITH SINGLE SPA .....</b> | <b>11</b> |
| <b>LAB 05: PERFORMANCE OPTIMIZATION.....</b>         | <b>13</b> |

## Disclaimers and Copyright Statement

### Disclaimer

WatzThis? takes care to ensure the accuracy and quality of this courseware and related courseware files. We cannot guarantee the accuracy of these materials. The courseware and related files are provided without any warranty whatsoever, including but not limited to implied warranties of merchantability or fitness for a particular purpose. The use of screenshots, product names and icons in the courseware is for editorial purposes only. No such use should be construed to imply sponsorship or endorsement of the courseware, nor any affiliation of such entity with WatzThis?.

### Third-Party Information

This courseware contains links to third-party websites that are not under our control, and we are not responsible for the content of any linked sites. If you access a third-party website mentioned in this courseware, you do so at your own risk. We provide these links only as a convenience, and the link's inclusion does not imply that we endorse or accept responsibility for the content on those third-party websites. Information in this courseware may change without notice and does not represent a commitment by the authors and publishers.

### Copyright

All rights reserved. No part of this publication may be reproduced, distributed, or transmitted in any form or by any means, including photocopying, recording, or other electronic or mechanical methods, without the prior expressed permission of the owners, except in the case of brief quotations embodied in critical reviews and certain other noncommercial uses permitted by copyright law. For permission requests, write to the owners at [info@watzthis.com](mailto:info@watzthis.com).

### Help us improve our courseware.

Please send your comments and suggestions via email to [info@watzthis.com](mailto:info@watzthis.com)

## Credits

### About the Author

Chris Minnick is a prolific published author, trainer, web developer, and founder of WatzThis, Inc. Minnick has overseen the development of hundreds of web and mobile projects for customers from small businesses to some of the world's largest companies, including Microsoft, United Business Media, Penton Publishing, and Stanford University.

Since 2001, Minnick has trained thousands of Web and mobile developers. In addition to his in-person courses, Chris has written and produced online courses for Ed2Go.com, Skillshare, O'Reilly Media, and Pluralsight.

Minnick has authored and co-authored books and articles on a wide range of Internet-related topics, including JavaScript, ReactJS, HTML, CSS, mobile apps, e-commerce, Web design, SEO, and security. His published books include JavaScript All-In-One For Dummies, Coding All-In-One For Dummies, ReactJS Foundations, JavaScript for Kids, Writing Computer Code, Coding with JavaScript For Dummies, Beginning HTML5 and CSS3 For Dummies, Webkit For Dummies, CIW eCommerce Certification Bible, and XHTML.

## Course Requirements

To complete the labs in this course, you will need:

- A computer with MacOS, Windows, or Linux.
- Access to the Internet.
- A modern web browser.
- Ability to install software globally.
- Experience with JavaScript and React.

## Lab 01: React Fundamentals

In this lab, we'll review fundamental concepts in React, including state management, component lifecycle, hooks, and class vs. function components. You'll use these concepts to implement new features for a real-time chat application.

- ☐ 1. If you haven't already, clone the repo at <https://github.com/chrisminnick/advanced-react>
- ☐ 2. Open the repository in VSCode. In it, you'll find a directory named **real-time-chat**. This is the starter project for this lab.
- ☐ 3. Open the **real-time-chat** directory. Inside it is a **client** directory and a **server** directory.
- ☐ 4. Open the **README** file and follow the installation instructions.
- ☐ 5. Once both the client and server are running, check the terminal window where you're running the server. You should see a message that the server is running, and that the server connected to the database.
- ☐ 6. Look at the client app in your browser. It should show the login page.
- ☐ 7. Click the signup link and fill it out (use a fake email address), then log in.
- ☐ 8. Open a different browser (such as Edge) and go to <http://localhost:3000> and sign up / login with that browser.
- ☐ 9. You should be able to chat with yourself. Fun!
- ☐ 10. Look through the server and client to see how they work. How would you improve them?
- ☐ 11. Look at the Future Features section of the README and think about how you might implement each one.
- ☐ 12. Pick one of the features and implement it. Make sure to use a separate git branch and commit your work frequently so you can always get back to something that works.
- ☐ 13. Repeat step 11 until the time is up for the lab.

## Lab 02: Routing and authentication

In this lab, you'll implement routes for user signup and authentication using an existing API.

- ☐ 1. Bootstrap a new React project in **/social-media** and name it client.

```
npx create-react-app client
```

- ☐ 2. Follow the instructions in social-media/server to install and run the server.
- ☐ 3. Install `react-router-dom` into **social-media/client** and copy code and components from the real-time-chat project to implement **/signup**, **/login**, and **/logout** routes. Hint: the API for the social-media project has a **/user** endpoint that's identical to the one in the real-time-chat project.
- ☐ 4. Make signup, login, and logout work and redirect the user to a restricted route when they successfully login.
- ☐ 5. Challenge: Pull data from the **/posts** endpoint and display posts on the restricted route users are redirected to.
- ☐ 6. Bonus Challenge: Create a **/post/:id** route that will display a single post
- ☐ 7. Bonus Challenge: Make a form that users can use to create new posts and hook it up. Hint: look in the **server/api** directory for sample HTTP Requests. Install the Rest Client VSCode extension (by Huachao Mao) for VSCode to run these.

## Lab 03: Implementing Redux

In this lab, you'll start with an existing non-redux app and convert it to use Redux.

- ☐ 1. Install the required dependencies: `redux`, `react-redux`, and `@reduxjs/toolkit`.
- ☐ 2. Create a **store** directory in your project's root directory.
- ☐ 3. Create a **store.js** file in the store directory.
- ☐ 4. In **store.js**, import `configureStore` from `@reduxjs/toolkit`.
- ☐ 5. Create a `rootReducer` that combines your reducers.

```
import { configureStore } from '@reduxjs/toolkit';
import { combineReducers } from 'redux';
import { authSlice } from '../features/auth/authSlice';
import { postsSlice } from '../features/posts/postsSlice';

const rootReducer = combineReducers({
 auth: authSlice.reducer,
 posts: postsSlice.reducer,
});
```

- ☐ 6. Use `configureStore` to create a store with your `rootReducer`.

```
export const store = configureStore({
 reducer: rootReducer,
});
```

- ☐ 7. Import the store from your store module into App.
- ☐ 8. Import `Provider` from `react-redux`.
- ☐ 9. Wrap your App component with the `Provider` component from `react-redux`, passing in your store as a prop.

```
function App() {
 return (
 <Provider store={store}>
 <div className="App container">
 <Routes />
 </div>
 </Provider>
);
}
```

- ☐ 10. Import `{BrowserRouter as Router}` from `react-router-dom` in `App.js` and wrap the div with `<Router>`.

```
function App() {
 return (
 <Provider store={store}>
 <Router>
 <div className="App container">
 <Routes />
 </div>
 </Router>
 </Provider>
)
}
```

```
};
```

- 11. Update the `ProtectedRoutes` component to use the `useSelector` hook to get the user property from the Redux state:

```
import { Navigate, Outlet } from 'react-router-dom';
import { useSelector } from 'react-redux';

export const ProtectedRoute = () => {
 const { user } = useSelector((state) => state.auth);

 // Check if the user is authenticated
 if (!user) {
 // If not authenticated, redirect to the login page
 return <Navigate to="/login" />;
 }

 // If authenticated, render the child routes
 return <Outlet />;
};
```

- 12. Update the `routes/index.js` file to look like this:

```
import { Route, Routes as RouterRoutes } from 'react-router-dom';
import { ProtectedRoute } from '../ProtectedRoutes';

import LoginPage from '../pages/LoginPage';
import SignupPage from '../pages/SignupPage';
import LogoutPage from '../pages/LogoutPage';
import HomePage from '../pages/HomePage';
import PostsPage from '../pages/PostsPage';

const Routes = () => {
 return (
 <RouterRoutes>
 <Route path="/" element={<HomePage />} />
 <Route path="/login" element={<LoginPage />} />
 <Route path="/logout" element={<LogoutPage />} />
 <Route path="/signup" element={<SignupPage />} />
 <Route path="/posts" element={<ProtectedRoute />}>
 <Route path="/posts" element={<PostsPage />} />
 </Route>
 </RouterRoutes>
);
};

export default Routes;
```

- 13. Create a **pages** directory in **src**. and create the components that you imported into **routes/index.js**. Copy over and rename any components that you have in **/components** already.
- 14. Create a **features** directory in **src** and subdirectories named **posts** and **auth**.

- ☐ 15. Create a file named **authSlice.js** in **features/auth/**.
- ☐ 16. Copy the code for **authSlice.js** from **/solutions/social-media-redux/client/features/auth/authSlice.js**
- ☐ 17. Update the `LoginPage`, `SignupPage`, and `LogoutPage` by referring to **/solutions/social-media-redux/client** in the GitHub repo.
- ☐ 18. Copy **api/api.js** from **solution/social-media-redux/client**.
- ☐ 19. Start the server (by running **npm start** in the **/server** directory).
- ☐ 20. Start the client app.
- ☐ 21. Test your app in the browser and fix any errors that prevent it from running.
- ☐ 22. Sign up for a new account, Log in, then log out to make sure it all works.
- ☐ 23. Install the Redux Dev Tool (if you don't have it already) and repeat the above steps with the Dev Tool open.
- ☐ 24. Add a form to the **Posts** page for adding new posts and connect it to Redux and the API.
- ☐ 25. Make the **Posts** page display a list of the current posts.
- ☐ 26. Make clicking a single post navigate to a new route, **posts/[id]** that displays just that post.
- ☐ 27. Implement or update additional features as time permits.

## Lab 04: Micro frontends with Single SPA

In this lab, you'll use the Single SPA framework to create a microfrontend.

- ☐ 1. In an empty directory that's not inside of any other Node project (no **package.json** at a higher level), open a new terminal window and invoke `create-single-spa`.

```
npx create-single-spa --moduleType root-config
```

- ☐ 2. Answer all the questions that `create-single-spa` asks, choosing the defaults whenever possible.
- ☐ 3. Run **npm start** in your new project and open a browser to <http://localhost:9000>.

You now have a root config and an example application. You'll see some instructions for what to do next in the sample application that's running at port 9000. Read through those instructions. We're going to use Single SPA to run two React applications and share dependencies between them.

- ☐ 4. Open a new terminal window and generate a single-spa application by running:

```
npx create-single-spa --moduleType app-parcel
```

- ☐ 5. When you're asked for a directory and a name for the application, name the directory something creative like 'app1' and the app 'my-first-app.'
- ☐ 6. Once it finishes, cd to your new directory and run **npm start**.
- ☐ 7. Open a browser and go to the localhost port that it gives you when it starts up. (probably localhost:8001).
- ☐ 8. Read through this page, but don't follow these instructions just yet.
- ☐ 9. Open **src/index.ejs** in your root config (not in your app1 subdirectory) and find the script element with `type="systemjs-importmap"`. Since all of our microfrontends will use React, we need to add React and ReactDOM to this import map.
- ☐ 10. Go to <https://cdnjs.com/libraries/react> and get the latest link for the React library (it should have **umd** in the URL) and add it to the **importmap**, then do the same for the ReactDOM library (you can just copy the same url and change "react" to "react-dom" in the URL).
- ☐ 11. When you're finished, your importmap should look like this:

```
<script type="systemjs-importmap">
 {
 "imports": {
 "single-spa": "https://cdn.jsdelivr.net/npm/single-spa@5.9.0/lib/system/single-spa.min.js",
 "react":
 "https://cdnjs.cloudflare.com/ajax/libs/react/18.2.0/umd/react.production.min.js",
 "react-dom": "https://cdnjs.cloudflare.com/ajax/libs/react-dom/18.2.0/umd/react-dom.production.min.js"
 }
 }
</script>
```

- ☐ 12. Open **src/microfrontend-layout.html** and find the `<route>` element. Add your new application as a 2nd application. You can get the value for the name property from the **package.json** file in your **app1** directory. For example:

```
<application name="app1/@minnick/my-first-app"></application>
```

- ☐ 13. In your root config's **src** directory, open **index.ejs** and add your application to the `importmap`. Note that there are two `importmaps` in the file, and you should add your application to both. Here's an example of what you should add:

```
"@minnick/my-first-project": "///localhost:8081/minnick-my-first-app.js",
```

- ☐ 14. Stop both your root config and your application and restart them. In your browser, you should now see a message saying that your application is mounted. It will look like this:

```
@minnick/my-first-app is mounted!
```

- ☐ 15. Remove the sample application from your **src/index.ejs** so your new application is the only one being rendered.
- ☐ 16. Create a second application and render that one in addition to the first.

## Lab 05: Performance Optimization

- ☐ 1. Use the React profiler tool and the Chrome Developer Tool Lighthouse tool to profile the social media app and identify slow components of the app.
- ☐ 2. Use techniques you've learned to improve the app's performance.